Practica 2

▼ Ejercicio 1.1

Veo que los protocolos presentados en la catedra huelen mal, sin una explicación no se entiende que hacen

```
//antes
public double lmtCrdt() {...
    //Retorna el límite de crédito del cliente

protected double mtFcE(LocalDate f1, LocalDate f2) {...
    //Retorna el monto facturado al cliente desde la fec
ha f1 a la fecha f2

private double mtCbE(LocalDate f1, LocalDate f2) {...
    //Retorna el monto cobrado al cliente desde la fecha
f1 a la fecha f2

//despues
public double getLimiteCredito() {}

public double getMontoFacturadoPeriodo(LocalDate fechaIn
icio, LocalDate fechaFin) {}

public double getMontoCobradoPeriodo(LocalDate fechaInic
io, LocalDate fechaFin) {}
```

▼ Ejercicio 1.2

Se realizo un cambio de responsabilidades, antes la persona le preguntaba al proyecto si participaba, lo que generaba un alto acoplamiento. Considero que el cambio es apropiado.

▼ Ejercicio 1.3

Analice el código que se muestra a continuación. Indique qué *code smells* encuentra y cómo pueden corregirse.

```
public void imprimirValores() {
    int totalEdades = 0;
    double promedioEdades = 0;
    double totalSalarios = 0;

    for (Empleado empleado : personal) {
        totalEdades = totalEdades + empleado.getEdad();
        totalSalarios = totalSalarios + empleado.getSala
rio();
    }
    promedioEdades = totalEdades / personal.size();

    String message = String.format("El promedio de las e
dades es %s y el total de salarios es %s",        promed
ioEdades, totalSalarios);

    System.out.println(message);
}
```

Se identifican varios code smells. En primer lugar, existe un error en el cálculo del promedio debido a una división entera, lo que provoca pérdida de precisión. Además, el método presenta bajo nivel de reutilización al depender directamente de la colección interna `personal`. También viola el principio de responsabilidad única, ya que realiza múltiples tareas: cálculo, formateo e impresión. Por último, no contempla el caso de lista vacía, lo que puede generar una división por cero.

Posible corrección:

```
public void imprimirValores(List<Empleado> empleados) {
    int totalEdades = 0;
    double promedioEdades = 0;
    double totalSalarios = 0;

    totalEdades = getTotalEdades(empleados);
```

```
    totalSalarios = getTotalSalarios(empleados);

    if(!empleados.size().isEmpty()){
        promedioEdades = (double) totalEdades / empleado
s.size();
    }

    String message = String.format("El promedio de las e
dades es %s y el total de salarios es %s", promedioEdade
s, totalSalarios);

    System.out.println(message);
}

private int getTotalEdades(List<Empleado> empleados){
    int totalEdades = 0;

    for (Empleado empleado : empleados){
        totalEdades += empleado.getEdad();
    }
    return totalEdades;
}

private double getTotalSalarios(List<Empleado> empleado
s){
    double totalSalarios = 0;

    for(Empleado empleado : empleados){
        totalSalarios += empleado.getSalario();
    }
    return totalSalarios;
}
```

▼ Ejercicio 2

Código original:

```
1   public class CharRing extends Object {
2       char[] source;
```

```
    int idx;

    public CharRing(String srcString) {
        char result;
        source = new char[srcString.length()];
        srcString.getChars(0, srcString.length(), source, 0);
        result = 0;
        idx = result;
    }

    public char next() {
        int result;
        if (idx >= source.length)
            idx = 0;
        result = idx++;
        return source[result];
    }
}
```

Código con error:

```
 public class CharRing extends Object {
    char[] source;
    int idx;

    public CharRing(String srcString) {
        char result;
        source = new char[srcString.length()];
        srcString.getChars(0, srcString.length(), source, 0);
        result = 0;
        idx = result;
    }

    public char next() {
        int result;
        if (idx >= source.length)
```

```
            idx = 0;
        result = idx++;
        return source[currentPosition];
    }
}
```

El error se produce porque el refactoring no se aplicó correctamente: se cambió el uso de la variable pero no su declaración. Rename Variable implica modificar todas las ocurrencias dentro del mismo scope. Una vez corregido, el código compila.

Código final:

```
 1   public class CharRing extends Object {
 2       char[] source;
 3       int idx;
 4
 5       public CharRing(String srcString) {
 6           char result;
 7           source = new char[srcString.length()];
 8           srcString.getChars(0, srcString.length(), so
urce, 0);
 9           result = 0;
10           idx = result;
11       }
12
13       public char next() {
14           int currentPosition;
15           if (idx >= source.length)
16               idx = 0;
17           currentPosition = idx++;
18           return source[currentPosition];
19       }
20   }
```